

M15 / PROTECTED ANSWERS

M15 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

- Read this first 8
- 1. DE15-01 - Cloud fundamentals for data engineers 8
 - One acceptable solved answer 8
 - Why this is acceptable 8
 - Common wrong answers 8
 - Validate your own attempt 8
 - Changed retry 8
- 2. DE15-02 - Identity, IAM and least privilege 8
 - One acceptable solved answer 8
 - Why this is acceptable 9
 - Common wrong answers 9
 - Validate your own attempt 9
 - Changed retry 9
- 3. DE15-03 - Azure Storage concepts and OneLake 9
 - One acceptable solved answer 9
 - Why this is acceptable 9
 - Common wrong answers 9
 - Validate your own attempt 9
 - Changed retry 10
- 4. DE15-04 - Fabric workspace and item model 10
 - One acceptable solved answer 10

Why this is acceptable 10

Common wrong answers 10

Validate your own attempt 10

Changed retry 10

5. DE15-05 - OneLake paths, files, tables and shortcuts 10

One acceptable solved answer 10

Why this is acceptable 10

Common wrong answers 11

Validate your own attempt 11

Changed retry 11

6. DE15-06 - Fabric Lakehouse and Delta tables 11

One acceptable solved answer 11

Why this is acceptable 11

Common wrong answers 11

Validate your own attempt 11

Changed retry 11

7. DE15-07 - Fabric notebooks 11

One acceptable solved answer 11

Why this is acceptable 12

Common wrong answers 12

Validate your own attempt 12

Changed retry 12

8. DE15-08 - Spark in Fabric 12

One acceptable solved answer	12
Why this is acceptable	12
Common wrong answers	12
Validate your own attempt	12
Changed retry	12
9. DE15-09 - Fabric Data Pipelines	13
One acceptable solved answer	13
Why this is acceptable	13
Common wrong answers	13
Validate your own attempt	13
Changed retry	13
10. DE15-10 - SQL analytics endpoint and warehouse choice	13
One acceptable solved answer	13
Why this is acceptable	13
Common wrong answers	13
Validate your own attempt	14
Changed retry	14
11. DE15-11 - Medallion architecture in Fabric	14
One acceptable solved answer	14
Why this is acceptable	14
Common wrong answers	14
Validate your own attempt	14
Changed retry	14

12. DE15-12 - Incremental loads and MERGE in cloud	14
One acceptable solved answer	14
Why this is acceptable	15
Common wrong answers	15
Validate your own attempt	15
Changed retry	15
13. DE15-13 - Permissions, connections and secrets	15
One acceptable solved answer	15
Why this is acceptable	15
Common wrong answers	15
Validate your own attempt	15
Changed retry	15
14. DE15-14 - Monitoring, reliability and cost awareness	16
One acceptable solved answer	16
Why this is acceptable	16
Common wrong answers	16
Validate your own attempt	16
Changed retry	16
15. DE15-15 - Git, deployment and environment separation	16
One acceptable solved answer	16
Why this is acceptable	16
Common wrong answers	16
Validate your own attempt	17

Changed retry	17
16. DE15-16 - End-to-end Fabric data engineering case	17
One acceptable solved answer	17
Why this is acceptable	17
Common wrong answers	17
Validate your own attempt	17
Changed retry	17
17. INT-01 - Runtime boundary	18
One acceptable solved answer	18
Why this is acceptable	18
Common wrong answers	18
Validate your own attempt	18
Changed retry	18
18. INT-02 - Control-plane vs data-plane failure	18
One acceptable solved answer	18
Why this is acceptable	18
Common wrong answers	18
Validate your own attempt	18
Changed retry	18
19. INT-03 - Security + deployment	19
One acceptable solved answer	19
Why this is acceptable	19
Common wrong answers	19

Validate your own attempt	19
Changed retry	19
20. INT-04 - Cost + correctness	19
One acceptable solved answer	19
Why this is acceptable	19
Common wrong answers	19
Validate your own attempt	19
Changed retry	20

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a saved closed-book attempt. These are solved examples, not learning-goal restatements.

1. DE15-01 - Cloud fundamentals for data engineers

One acceptable solved answer

Six-box design: (1) operational orders DB/source; (2) Fabric Data Pipeline as control/orchestration; (3) OneLake/Lakehouse Files Bronze landing; (4) Fabric notebook/Spark transform; (5) Lakehouse Delta Silver/Gold tables; (6) SQL analytics endpoint/BI consumer. Identity is the pipeline/notebook execution identity and workspace permissions; monitoring is pipeline run history + notebook/Spark evidence + reconciliation metrics.

Storage boundary: OneLake Files/Tables. Compute boundary: pipeline control plane and Spark notebook. Identity boundary: workspace/item connection/execution principal. Monitoring boundary: run ID, source/committed watermark, counts and paid revenue.

Why this is acceptable

It uses only two Fabric compute item types (Pipeline + Notebook/Spark), separates durable storage from execution, and gives each boundary observable evidence.

Common wrong answers

Wrong: putting secrets inside the notebook; calling the SQL endpoint a separate copy of the data; adding Warehouse/Eventstream only to make the diagram look advanced.

Validate your own attempt

A reviewer should be able to point at every box and answer: what is durable here, what executes here, who authorizes it, and what proves success?

Changed retry

Change the source to on-premises SQL Server. Add only the required connectivity/gateway/private-network concern; keep the storage/transform/serving contracts stable.

2. DE15-02 - Identity, IAM and least privilege

One acceptable solved answer

Developer: workspace Contributor in DEV; must not change PROD workspace roles or secrets.

Data consumer: Viewer/read permission to approved Gold/semantic/SQL surfaces; must not edit notebooks, pipelines or tables.

Deployment automation: dedicated workload identity/service principal with only the permissions needed to deploy supported items to target workspaces; must not be Workspace Admin/Owner by default or read unrelated data.

Production support: Viewer plus the narrow operational permissions required to inspect runs/logs and rerun approved jobs; must not change security assignments or production code directly outside the release path.

Why this is acceptable

Each principal has a concrete role and scope. The role may need item/data-plane permissions too; workspace role alone is not a complete authorization model.

Common wrong answers

Wrong: Contributor for every persona; shared personal accounts; giving deployment automation permanent Admin because it is easier.

Validate your own attempt

For each persona write principal -> role -> scope -> allowed action -> prohibited action. If any assignment lacks a scope, least privilege cannot be reviewed.

Changed retry

Add a two-week contractor who must diagnose a Gold SELECT issue without editing any item. Give only time-bounded read/diagnostic access.

3. DE15-03 - Azure Storage concepts and OneLake

One acceptable solved answer

Raw CRM JSON: Files/bronze/crm/... because raw/semi-structured evidence should remain file-oriented and replayable.

Cleaned customer Delta: Tables/silver_customers because it needs managed table semantics, schema and queryability.

Curated customer_daily: Tables/gold_customer_daily because it is a serving contract for consumers and should be queryable from supported table surfaces.

Why this is acceptable

Files and Tables are separate contracts even though both live in OneLake.

Common wrong answers

Wrong: placing raw evidence directly in a curated table and discarding source bytes; calling every OneLake path a shortcut.

Validate your own attempt

Confirm the path expresses layer + dataset and that a replay can start from Bronze without depending on a cleaned file.

Changed retry

Move Bronze to external ADLS Gen2. Keep Silver/Gold in Fabric and explain identity/path/shortcut changes.

4. DE15-04 - Fabric workspace and item model

One acceptable solved answer

DEV inventory: orders_lh Lakehouse; pl_orders_daily Pipeline; nb_transform_orders Notebook; env_orders Environment.

Dependencies: pl_orders_daily -> nb_transform_orders -> orders_lh; notebook -> environment; pipeline connection/identity -> source; SQL endpoint is generated from the lakehouse for read/query serving.

Why this is acceptable

The inventory makes dependencies explicit and separates item identity from data identity.

Common wrong answers

Wrong: one unlabeled portal screenshot; mixing DEV and PROD items in one diagram without deployment boundaries.

Validate your own attempt

Every arrow must mean a real dependency or data/control flow. Remove any item that has no stated responsibility.

Changed retry

Split into DEV and PROD workspaces and list which connection/lakehouse/environment bindings must change after deployment.

5. DE15-05 - OneLake paths, files, tables and shortcuts

One acceptable solved answer

Daily exchange rates: copy if your pipeline requires a durable independent snapshot for audit/replay; shortcut only if source availability/freshness is acceptable.

Immutable regulatory snapshot: copy/preserve under your retention control because independent evidence is the requirement.

4-TB shared customer dimension: prefer shortcut when ownership/security/availability of the source are acceptable, because copying 4 TB duplicates storage and refresh work.

Why this is acceptable

The decision follows ownership, retention, freshness and failure-domain requirements rather than product novelty.

Common wrong answers

Wrong: shortcut everything because it is cheaper; copy everything because it feels safer.

Validate your own attempt

Write source owner, required retention, tolerated outage, freshness and copy cost for each dataset. The decision should follow those columns.

Changed retry

Require PROD to survive a four-hour source-workspace outage; reconsider the customer-dimension shortcut.

6. DE15-06 - Fabric Lakehouse and Delta tables

One acceptable solved answer

Files/bronze/orders/... keeps raw order files; Tables/silver_orders and Tables/silver_customers are Delta tables written/merged by Spark; Tables/gold_daily_sales is curated Delta. The lakehouse SQL analytics endpoint reads/query-serves managed Delta tables; it is not the same as using a Fabric Warehouse for writable SQL transformation patterns.

Why this is acceptable

It names the durable table owner and the read surface separately.

Common wrong answers

Wrong: claiming the SQL endpoint is a second warehouse copy; writing mutable curated data only as CSV.

Validate your own attempt

Prove which engine performs writes and which surface is read-only/query-serving for this design.

Changed retry

Finance now needs stored procedures and writable SQL transformations; justify adding Warehouse only for that requirement.

7. DE15-07 - Fabric notebooks

One acceptable solved answer

Parameters: source_path, run_date, target_layer; mark defaults in a parameter cell when the notebook is called by a pipeline.

Validation before write: source exists; schema/required keys; parseable timestamps; finite/nonnegative amount; allowed status; duplicate business-key/version policy; row/key reconciliation. Retrieve credentials through managed connections/Key Vault/workload identity rather than a literal token in the cell.

Phases: parameters -> read -> validate -> transform -> write -> reconcile -> emit run evidence.

Why this is acceptable

The notebook can be automated because execution no longer depends on hidden human cell order or embedded secrets.

Common wrong answers

Wrong: hard-coded /mnt/. . . path, literal API token, or writing output before validation.

Validate your own attempt

A fresh session should run top-to-bottom with only parameters changed. Intentionally pass `target_layer="bad"` and verify it fails before any write.

Changed retry

Pipeline passes `run_date` and `target_layer`. Reject any target layer outside bronze/silver/gold.

8. DE15-08 - Spark in Fabric

One acceptable solved answer

DEV/PROD evidence checklist: Fabric runtime name/version; Spark version; Python version; Delta version; attached Fabric Environment; library/project dependency version (for example pandas or delta-related package if explicitly used); workspace/item; run ID/time. Record the same fields in DEV and PROD before attributing a behavior change to code.

Why this is acceptable

Runtime is part of reproducibility evidence.

Common wrong answers

Wrong: assuming the local M13 Spark/Delta versions must be forced into Fabric Runtime; writing only "Spark 4".

Validate your own attempt

Compare DEV and PROD evidence side by side and identify every mismatch before changing code.

Changed retry

DEV opts into Runtime 2.0 while PROD is older. Require compatibility, data-result, performance and dependency tests before promotion.

9. DE15-09 - Fabric Data Pipelines

One acceptable solved answer

Activities: Lookup/read committed watermark -> Copy/load candidate change set to durable Bronze -> Notebook transform/MERGE Silver -> validation/reconciliation -> Gold refresh -> commit new watermark.

Retry safety: lookup is read-only; copy is safe when landing path/run ID is immutable/idempotent; MERGE is safe only with deterministic version/key rules; validation is read-only; watermark commit is last and should not repeat a harmful side effect.

Failure: source timeout before durable copy leaves committed watermark unchanged; retry from the old committed boundary.

Why this is acceptable

Candidate progress is distinct from committed progress.

Common wrong answers

Wrong: save watermark immediately after source query; call any green Copy activity “correct”.

Validate your own attempt

For each activity mark durable input, durable output, retry behavior and whether it can advance progress.

Changed retry

Add a source timeout after candidate lookup but before copy and show why committed state does not change.

10. DE15-10 - SQL analytics endpoint and warehouse choice

One acceptable solved answer

Ad-hoc SELECT over Lakehouse Delta: SQL analytics endpoint.

Spark MERGE: Spark/notebook against the Lakehouse Delta table.

Stored-procedure-based SQL transformation: Fabric Warehouse when writable SQL/warehouse features are required.

Power BI consumption: semantic/Direct Lake or supported SQL/warehouse serving surface depending the model; do not duplicate authoritative Gold just for the editor.

Why this is acceptable

The choice follows write model and serving requirement.

Common wrong answers

Wrong: treating every SQL editor as the same writable engine.

Validate your own attempt

Name which engine owns writes and which table is authoritative.

Changed retry

Add a writable SQL staging requirement but keep one authoritative Gold contract.

11. DE15-11 - Medallion architecture in Fabric

One acceptable solved answer

Bronze: grain = one landed source record/event; key = source/run identity + business key when available; check = file/row/hash/schema presence; consumers = engineering/replay only.

Silver: grain = one current order per order_id; key = order_id; check = required fields, valid status/amount, deterministic latest-version rule; consumers = downstream engineering/controlled analytics.

Gold: grain = one business metric row per reporting date (or declared serving grain); key = reporting date/dimension keys; check = reconcile paid revenue/order counts to Silver; consumers = BI/analysts.

Why this is acceptable

Each layer has a distinct trust/consumer contract.

Common wrong answers

Wrong: Bronze=bad, Silver=good, Gold=best without grain/keys/validation.

Validate your own attempt

You should be able to rebuild Silver/Gold from preserved Bronze and explain every loss/filter.

Changed retry

Add immutable monthly regulatory snapshots without freezing every Gold table.

12. DE15-12 - Incremental loads and MERGE in cloud

One acceptable solved answer

Selection: read changes after the committed boundary, using overlap/CDC as required by the source contract rather than blindly trusting one timestamp.

MERGE key: customer_id; apply only a strictly newer version. Equal-version identical data is replay/no-op; equal-version conflicting data is surfaced, not arbitrarily ranked.

Deletes: if is_deleted=true, either tombstone/soft-delete in Silver or delete physically only if the contract allows; document which.

Commit point: after durable target write + validation/reconciliation. Never move the checkpoint backward on a normal replay.

Why this is acceptable

This matches the repaired M15 simulator and separates change detection, target precedence and progress ownership.

Common wrong answers

Wrong: ROW_NUMBER arbitrary winner for timestamp ties; watermark updated before target validation; older replay overwrites newer state.

Validate your own attempt

Run the supplied simulator scenarios: first load, identical replay, newer, stale, equal-version conflict, failed validation, interrupted write/recovery. Revenue after stale replay must remain 460.00.

Changed retry

Assume the source can correct updated_at backward. Replace the simple watermark contract with CDC/sequence or a bounded overlap plus deterministic source version.

13. DE15-13 - Permissions, connections and secrets

One acceptable solved answer

May live in source control (if not sensitive): logical workspace/environment alias, table name, public API base URL, Key Vault secret name.

Must not live in source control: API token, secret value, passwords, private connection strings/keys. Production resource IDs/endpoints may also be environment configuration rather than hard-coded code even when not secret.

Why this is acceptable

Secret identifiers and secret values are different; configuration placement also affects deployment portability.

Common wrong answers

Wrong: treating a Key Vault secret name as the secret itself, or assuming a nonsecret PROD URL should be hard-coded in every notebook.

Validate your own attempt

Search the repo for token-like values and prove runtime retrieves secret material from a managed location/identity.

Changed retry

Replace client secret authentication with managed/federated identity where supported; identify the removed secret and remaining authorization role.

14. DE15-14 - Monitoring, reliability and cost awareness

One acceptable solved answer

Dashboard spec: (1) last successful run age - alert > 75 min for hourly pipeline; (2) source rows vs landed rows difference - alert nonzero unless explained quarantine; (3) Silver unique-key reconciliation - alert duplicates/nonzero mismatch; (4) quarantine rate - warn >1%, critical >5%; (5) duration - warn >2x 7-day median; (6) paid-revenue or business total variance - alert outside agreed tolerance; optionally capacity/compute utilization for cost diagnosis.

Why this is acceptable

It mixes control-plane, data-quality and business evidence instead of showing only “pipeline success”.

Common wrong answers

Wrong: alert only on failed status; scale capacity before checking data shape/skew.

Validate your own attempt

For each metric specify owner, source, baseline/threshold and first diagnostic action.

Changed retry

Duration doubles while volume/capacity are unchanged: inspect Spark plan, skew/partition sizes and source/file layout before resizing capacity.

15. DE15-15 - Git, deployment and environment separation

One acceptable solved answer

Git: notebook/source code, supported pipeline/item definitions, tests, documentation, IaC/config templates.

Environment-specific: credentials, workspace/item IDs, connections, lakehouse bindings, secret references, capacity choice, PROD schedules and data.

Smoke test: after deploy, run a tiny known input/SELECT and prove the target points to PROD resources and returns the expected count/key/metric.

Why this is acceptable

Definitions are versioned while persistent data and secrets remain owned by the target environment.

Common wrong answers

Wrong: copying DEV secrets/bindings into PROD or editing source code manually after every deployment.

Validate your own attempt

Compare a logical binding table DEV->TEST->PROD and verify no PROD item references DEV resource IDs.

Changed retry

Target lakehouse/connection names differ: use deployment rules/config mapping, then smoke-test the bound target.

16. DE15-16 - End-to-end Fabric data engineering case

One acceptable solved answer

Concrete design: source orders -> pl_m15_orders -> OneLake/Lakehouse Files Bronze -> nb_m15_orders validation/MERGE -> silver_orders -> gold_order_summary -> SQL analytics endpoint. Use workspace/item identity and managed connection/Key Vault rather than embedded secret. Store committed watermark in m15_pipeline_control and update it only after Gold reconciliation.

Expected supplied-data result: after initial + changes, 5 unique Silver orders and paid revenue 460.00. Deliberately set expected revenue to 999.00 to create a failed validation; prove committed watermark does not advance, correct to 460.00, rerun, and reconcile.

Monitoring: run ID, input rows, Silver count, Gold revenue, quarantine count, committed watermark, duration. Deployment: code/definitions promoted; bindings/connections/environment-specific values mapped to target.

Why this is acceptable

This is reproducible from the new practice/FABRIC_GUIDED_BUILD.md, not just an architecture paragraph.

Common wrong answers

Wrong: a screenshot-only project, committing watermark before validation, adding extra Fabric items with no requirement.

Validate your own attempt

A beginner should be able to reproduce the five-order/460.00 result and explain failure/recovery from the guide.

Changed retry

Keep product data external via OneLake shortcut and add a Fabric Warehouse only if finance truly needs writable SQL Gold transformations.

17. INT-01 - Runtime boundary

One acceptable solved answer

Do not force the M13 local Spark/Delta versions into Fabric because Fabric is a managed runtime bundle. Use the runtime attached to the Fabric Environment/workspace, record Spark/Python/Delta evidence, and test the project against that supported bundle. Local M13 versions are a separate learning/runtime baseline.

Why this is acceptable

Managed-service compatibility is controlled by the service runtime, not by copying a local pip coordinate into the platform.

Common wrong answers

Wrong: “same course, therefore same Spark/Delta must be installed”.

Validate your own attempt

Record DEV/PROD runtime evidence and run compatibility tests before promotion.

Changed retry

Assume PROD is one Fabric runtime behind DEV; define the promotion test matrix.

18. INT-02 - Control-plane vs data-plane failure

One acceptable solved answer

Example: pipeline is green because every activity returned success, but a wrong watermark predicate selected zero rows. Another example: a Copy and notebook succeed but duplicate business keys double paid revenue. The control plane is green while semantic data is wrong.

Why this is acceptable

Orchestration status proves execution state, not business correctness.

Common wrong answers

Wrong: “green pipeline means correct data”.

Validate your own attempt

Add source-vs-landed counts, unique-key reconciliation and a business total such as paid revenue.

Changed retry

Create a green run that intentionally processes zero rows and explain which data-plane alert must catch it.

19. INT-03 - Security + deployment

One acceptable solved answer

A notebook definition can deploy successfully but fail with 403 because the PROD execution identity, connection, Key Vault data-plane role, lakehouse/table permission or target binding differs from DEV. Diagnose the identity actually used at runtime and its target-scope permissions before changing notebook logic.

Why this is acceptable

Deployment of metadata does not imply authorization of runtime data-plane calls.

Common wrong answers

Wrong: adding broader Contributor/Admin to “fix” the 403 without identifying the denied resource/action.

Validate your own attempt

Capture principal, target resource, denied action, role assignment scope and connection binding.

Changed retry

Replace the failing secret-based principal with workload identity and state the exact target roles still required.

20. INT-04 - Cost + correctness

One acceptable solved answer

Use incremental loading instead of full reload when the source/change contract is reliable. Preserve overlap/CDC for late changes, idempotent MERGE, reconciliation and checkpoint-last semantics. This reduces scanned/copied/processed data without weakening correctness. Other acceptable examples include partition pruning or avoiding an unnecessary 4-TB copy with a shortcut when its availability contract is acceptable.

Why this is acceptable

The optimization reduces work while retaining correctness controls.

Common wrong answers

Wrong: reduce cost by removing validation, shrinking overlap below the late-data SLA, or skipping failed partitions.

Validate your own attempt

Show before/after work volume and prove the same reconciliation/replay behavior still passes.

Changed retry

Assume late updates can arrive two days late; tune the incremental strategy without losing them.